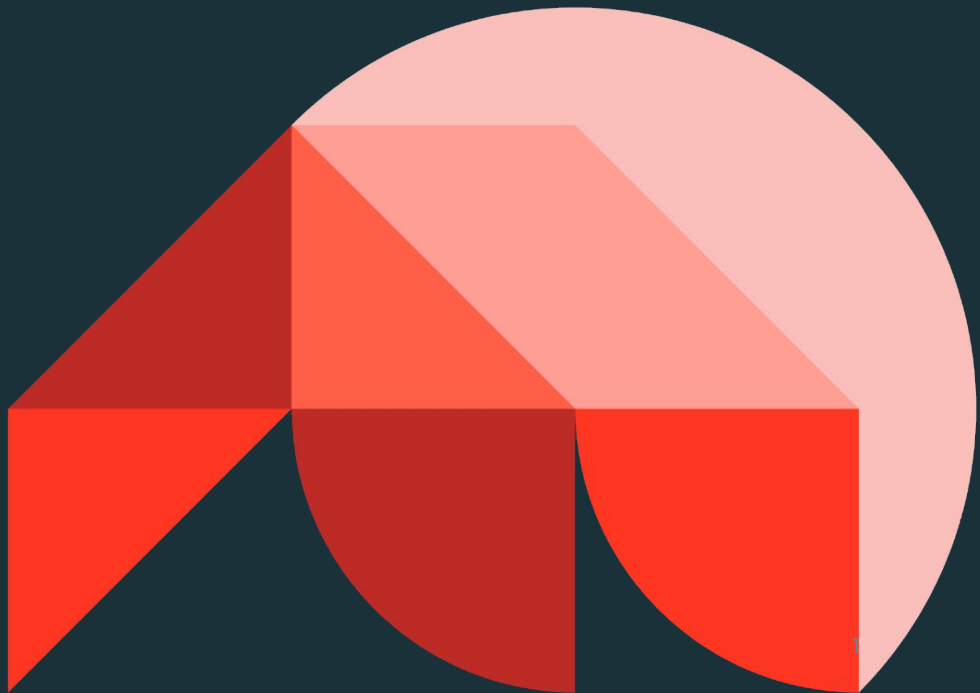




Genie Spaces

Implementation Guide

Shared Technical Services
Last updated April 2026



Agenda

-  **Pre-Flight: Platform & Security Setup** — Infrastructure, permissions, compliance controls
-  **Data Engineering for Genie** — Schema design, naming, annotations, metric views
-  **Multi-Room Architecture Patterns** — Domain decomposition for 100+ dataset estates
-  **Building Your First Genie Space** — Step-by-step with code at every stage
-  **Knowledge Store Mastery** — SQL expressions, example queries, joins, entity matching
-  **Advanced Curation Techniques** — Trusted assets, UDFs, parameterized queries, clarifications
-  **Benchmark-Driven Development** — Measuring accuracy, iterating systematically
-  **Sandbox → Production Playbook** — Phased rollout, CI/CD, cloning, monitoring cadence
-  **Operational Runbook** — Monitoring, feedback loops, continuous improvement
-  **Quick-Reference Cheat Sheets** — Limits, decision matrices, instruction hierarchy
-  **References** — All sources cited in this guide

Pre-Flight: Platform & Security Setup

Infrastructure Checklist

Requirement	Specification	Action
SQL Warehouse	Pro or Serverless (Serverless strongly recommended)	Provision via workspace admin
Unity Catalog	All Genie data must be UC-registered	Verify tables are in UC, not hive_metastore
Partner-Powered AI	Must be ON at account AND workspace level	Account admin > Settings > AI Features
Network	Private Link, VPC peering per enterprise policy	Verify SQL warehouse is reachable
Workspace entitlement	Databricks SQL entitlement for space creators	Admin console > Users > Entitlements



Permission Model — The Dual Credential Architecture

Genie uses a unique dual-credential model that is critical to understand for enterprise deployments:

Credential Type	Used For	Implication
Author's compute credentials	SQL execution on the warehouse	Author must have CAN USE on warehouse
User's data credentials	Data access via Unity Catalog	Row filters + column masks are enforced per user

This means: End users do NOT need direct warehouse permissions. They only need CAN VIEW/CAN RUN on the Genie space plus SELECT on the underlying UC objects. The author's compute credentials are embedded — all queries run under the author's warehouse access but the user's data access.

AVOID: The space author's compute credentials are permanently embedded. If the author leaves the organization or loses warehouse access, the space breaks. Designate a service account or shared admin as the space author for production spaces.



Permission Levels — Quick Reference

Level	Who	Can Do
CAN MANAGE	Space admins	Everything: configure, share, delete, monitor, benchmark
CAN EDIT	Data analysts / curators	Modify instructions, add example queries, run benchmarks
CAN RUN	Business users	Ask questions, get responses, provide feedback
CAN VIEW	Observers	Read space configuration and metadata only



Enabling Genie — Step by Step

Step 2.1: Enable Partner-Powered AI

```
# Via Databricks CLI
databricks account settings update-ai-features \
--setting-name partner-powered-ai \
--enabled true
```

```
# Verify at workspace level
databricks workspace-conf get-status \
--keys enableAIBIGenie
```



Enabling Genie — Step by Step

Step 2.2: Provision Serverless SQL Warehouse

```
databricks warehouses create \  
--name "genie-warehouse" \  
--cluster-size "Small" \  
--warehouse-type "SERVERLESS" \  
--auto-stop-mins 15 \  
--enable-serverless-compute true
```



Enabling Genie — Step by Step

Step 2.3: Grant Permissions

```
-- Grant warehouse access to space author
GRANT CAN USE ON WAREHOUSE `genie-warehouse`
TO `genie-admin@company.com`;

-- Grant data access to end users (via group)
GRANT USE CATALOG ON CATALOG gold TO `genie-users`;
GRANT USE SCHEMA ON SCHEMA gold.finance TO `genie-users`;
GRANT SELECT ON SCHEMA gold.finance TO `genie-users`;
```



Data Engineering for Genie

—

The #1 Rule: Naming Determines Accuracy

Field testing across dozens of enterprise deployments has established a clear pattern: table and column naming is the single largest factor in Genie accuracy. Clear naming alone can take you from 0% to 50%+ accuracy before any curation.

Before (0% accuracy)	After (50%+ accuracy)	Why It Matters
cmp	campaigns	Genie maps 'campaign' questions to correct table
uid_seq	campaign_id	Genie identifies this as a primary key
label_txt	campaign_name	Genie uses for GROUP BY and display
num_val	cost_usd	Genie generates SUM(cost_usd) for cost questions
proc_ts	event_timestamp	Genie applies date filters correctly
stat_cd	status_code	Genie matches status-related questions

ACTION: If renaming production tables is not feasible, create aliased views:

```
CREATE VIEW gold.finance.campaigns AS SELECT uid_seq AS campaign_id, label_txt AS campaign_name, ... FROM raw.marketing.cmp;
```



Gold Layer Design Principles

Genie works best with gold-layer, business-ready tables. Apply these principles during data engineering:

- **Denormalize aggressively** — Pre-join dimension tables into facts. Every join Genie must infer is a potential failure point.
- **Pre-calculate derived metrics** — Fiscal quarter, YTD totals, rolling averages, status flags — materialize them. Do not rely on Genie to derive `fiscal_quarter` from a raw date.
- **Filter at source** — Remove test data, internal records, deprecated rows during ETL. Do not use Genie instructions as a data filter — they are unreliable for this purpose.
- **Use metric views** — Encode shared business logic (revenue = `SUM(amount) - SUM(refunds)`) in metric views. Genie inherits these definitions as a single source of truth.



Unity Catalog Annotations — Complete Examples

Annotate every table and column before creating a Genie space. These annotations are the foundation of Genie's understanding.

Table Descriptions (Mission Statement Format)

```
ALTER TABLE gold.finance.daily_balances SET TBLPROPERTIES ('comment' = 'Daily end-of-day account balances for all active accounts. One row per account per business day. Use for: balance trends, ADB calculations, liquidity analysis. Granularity: account_id + balance_date. Updated: daily at 06:00 UTC via ETL pipeline.');
```

```
ALTER TABLE gold.finance.transactions SET TBLPROPERTIES ('comment' = 'All completed financial transactions across retail and commercial accounts. One row per transaction. Use for: volume analysis, revenue calculations, fraud detection queries. Excludes pending and reversed txns. Granularity: transaction_id (unique).');
```



Unity Catalog Annotations — Complete Examples

Column Descriptions (Disambiguate Everything)

```
ALTER TABLE gold.finance.transactions
  ALTER COLUMN amount SET COMMENT
  'Transaction amount in USD. Positive = credit to account,
  negative = debit. Use SUM(amount) for net flow.
  Use SUM(ABS(amount)) for gross volume.';
```

```
ALTER TABLE gold.finance.transactions
  ALTER COLUMN transaction_type SET COMMENT
  'Type of transaction. Values: PURCHASE, PAYMENT,
  TRANSFER, FEE, INTEREST, REWARD. Use for filtering
  and grouping by transaction category.';
```

```
ALTER TABLE gold.finance.transactions
  ALTER COLUMN created_at SET COMMENT
  'Timestamp when the transaction was initiated, in UTC.
  Use for time-series analysis. For settlement date,
  use settled_at column instead.';
```



Unity Catalog Annotations — Complete Examples

Primary & Foreign Keys

```
-- Primary keys
ALTER TABLE gold.finance.accounts
  ADD CONSTRAINT pk_accounts PRIMARY KEY (account_id);
ALTER TABLE gold.finance.transactions
  ADD CONSTRAINT pk_txn PRIMARY KEY (transaction_id);
ALTER TABLE gold.finance.customers
  ADD CONSTRAINT pk_cust PRIMARY KEY (customer_id);

-- Foreign keys (Genie uses these for automatic join paths)
ALTER TABLE gold.finance.transactions
  ADD CONSTRAINT fk_txn_account
  FOREIGN KEY (account_id)
  REFERENCES gold.finance.accounts(account_id);

ALTER TABLE gold.finance.accounts
  ADD CONSTRAINT fk_acct_customer
  FOREIGN KEY (customer_id)
  REFERENCES gold.finance.customers(customer_id);
```

RESULT: With clear naming + descriptions + PK/FK constraints, expect 40-55% benchmark accuracy before any Genie-specific curation. This is your foundation.



Multi-Room Architecture Pattern

—

When One Room Is Not Enough

Genie has a hard limit of 30 tables per space. Even below that limit, field testing shows that spaces with more than 10–12 tables begin to degrade in accuracy. For enterprises with 100+ datasets, multi-room architecture is mandatory.

Signal	Action
>30 tables needed	Multi-room is mandatory (hard limit)
>12 tables in a domain	Consider pre-joining into views to reduce count
Multiple business units querying different data	Separate rooms per BU
Different security profiles (PII vs. non-PII)	Separate rooms for compliance
Conflicting vocabulary across domains	Separate rooms to avoid instruction conflicts
Benchmark accuracy plateaus below 70%	Too many tables may be diluting context



Domain Decomposition Strategy

Organize your data estate into coherent business domains. Each domain becomes one Genie space:

Domain	Tables	Target Audience	Complexity
Platform FinOps	billing, compute, jobs, warehouses (5-8)	Platform Engineering	Low — system tables
Operational Metrics	pipeline health, SLAs, data quality (5-7)	Data Engineering	Low-Medium
Marketing Analytics	campaigns, conversions, channels (5-8)	Marketing Team	Medium
Customer Analytics	segments, balances, products (6-8)	Business Analysts	High (PII)
Transaction Analysis	transactions, merchants, fraud (5-7)	Fraud/Ops Analysts	High (PCI)
Risk Analytics	risk scores, exposures, portfolios (8-10)	Risk Analysts	High (regulatory)
Regulatory Reporting	capital ratios, compliance, audit (5-7)	Compliance Team	High



Multi-Agent Orchestrator Pattern

For questions that span multiple domains, build a Supervisor Agent using Databricks Apps + the Genie Conversation API:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import (
    ChatMessage, ChatMessageRole
)

w = WorkspaceClient()

# Domain-to-space routing map
SPACES = {
    "finops": {"id": "abc123", "desc": "Cost, billing, compute"},
    "customer": {"id": "def456", "desc": "Accounts, balances"},
    "transactions": {"id": "ghi789", "desc": "Payments, fraud"},
}

def classify_domain(question: str) -> str:
    """Use Foundation Model API to route the question."""
    domains_str = "\n".join(
```



Multi-Agent Orchestrator Pattern

```
f"- {k}: {v['desc']}" for k,v in SPACES.items()
)
resp = w.serving_endpoints.query(
    name="databricks-meta-llama-3-3-70b-instruct",
    messages=[ChatMessage(
        role=ChatMessageRole.USER,
        content=f"Classify this question into one domain:\n"
                f"{domains_str}\n\nQuestion: {question}"
    )],
    max_tokens=20
)
return resp.choices[0].message.content.strip().lower()

def ask_genie(question: str) -> dict:
    """Route question to correct Genie space."""
    domain = classify_domain(question)
    space_id = SPACES[domain]["id"]
    response = w.genie.start_conversation_and_wait(
        space_id=space_id, content=question
    )
    return {
        "domain": domain,
        "response": response,
        "sql": response.attachments[0].query.query
        if response.attachments else None
    }
```

ACTION: Deploy this as a Databricks App with a Streamlit or Gradio frontend. The app becomes the single entry point for all Genie queries, routing transparently to the correct domain space.



Building Your First Genie Space

Pre-Build Checklist

- 5-10 gold-layer tables selected, annotated with descriptions
- All PK/FK constraints defined in Unity Catalog
- Column descriptions written for every column in selected tables
- Unnecessary columns identified (will hide in Genie)
- Serverless SQL warehouse provisioned and accessible
- 3-5 SMEs identified with 20-30 representative questions
- Partner-Powered AI enabled at account + workspace level



Step 5.1: Create the Space

Navigate to **Genie** in the sidebar → **New** → Select your gold-layer tables → Click **Create**.

AVOID: Do NOT share the space yet. Complete all curation steps (5.2 through 5.7) before giving anyone access.

Step 5.2: Configure Settings

```
Title:      FinOps Cost Intelligence
Description: Ask questions about Databricks platform
            costs, compute usage, job performance,
            and warehouse utilization. Covers all
            workspaces and SKUs.
Warehouse:  genie-warehouse (Serverless)
Sample Qs:  - What are our top 10 most expensive jobs?
            - Show cost trends for the last 6 months
            - Which workspaces have the highest DBU usage?
```

Step 5.3: Hide Irrelevant Columns

Go to **Configure > Data** → Click each table → Uncheck columns that should not be queryable (internal IDs, ETL metadata, audit columns). This reduces token consumption and prevents confusing results.



Step 5.4: Add Column Synonyms

```
Column: sku_name
  Synonyms: SKU, product, compute type, billing SKU

Column: usage_quantity
  Synonyms: DBUs, usage, consumption, units

Column: list_price
  Synonyms: rate, unit price, DBU price, cost per DBU
```

Step 5.5: Enable Entity Matching

For categorical columns (status codes, product types, region names), enable entity matching so Genie can match user terms to actual data values:

```
Configure > Data > [table] > [column] > Enable Entity Matching
```

Good candidates:

sku_name	→ maps 'jobs compute' to 'JOBS_COMPUTE'
workspace_name	→ maps 'prod workspace' to 'prod-analytics'
billing_status	→ maps 'active' to 'ACTIVE'
region	→ maps 'us east' to 'us-east-1'

Limits:

- Max 120 columns with entity matching enabled per space
- Max 1,024 distinct values per column
- Max 127 characters per value
- Disabled for tables with row filters or column masks

Step 5.6: Define Join Relationships

Configure > Joins > Add Join

Join 1:

Left: gold.finops.billing_usage

Right: gold.finops.warehouses

Condition: billing_usage.warehouse_id = warehouses.warehouse_id

Cardinality: Many-to-One

Join 2:

Left: gold.finops.job_runs

Right: gold.finops.clusters

Condition: job_runs.cluster_id = clusters.cluster_id

Cardinality: Many-to-One

ACTION: Even if you defined PK/FK in Unity Catalog, explicitly add joins in the Genie knowledge store. This gives Genie both the constraint metadata AND the explicit instruction, providing the highest confidence for multi-table queries.



Knowledge Store Mastery

The Curation Hierarchy — Where to Put What

This is the single most important decision framework for Genie curation. Always start at the top and only move down when necessary:

Priority	Method	Limit	When to Use
1 ★★★	Data modeling (pre-join, pre-calc, filter)	No limit	Universal rules that should never vary
2 ★★★	UC metadata (names, descriptions, PK/FK)	No limit	Every table and column, always
3 ★★■	SQL expressions (measures, filters, dims)	200/space	Named business metrics and common filters
4 ★★■	Example SQL queries	Part of 100	Complex multi-step queries, metric definitions
5 ★★■	SQL functions / Trusted Assets	Part of 100	Calculations that must never vary
6 ★★■	Join definitions	No limit	All table relationships
7 ★■■	Entity matching / Value dictionaries	120 cols	Categorical columns with known values
8 ★■■	Column synonyms	No limit	Business terms that differ from column names
9 ■■■	General text instructions	1 block	LAST RESORT — global behavioral guidance only

AVOID: The most common curation mistake: putting everything in general text instructions. Text instructions should contain **less than 20%** of your total curation effort. The other 80% should be in SQL expressions, example queries, and metadata.



SQL Expressions — Your Most Powerful Tool

SQL expressions define reusable business logic that Genie can apply to any query.
Three types:

Measures (KPIs) — Aggregation Formulas

```
Name:      Total Revenue
SQL:       SUM(amount) - SUM(refund_amount)
Synonyms:  revenue, total sales, gross income
Instruction: Use when users ask about revenue, sales,
            or income. Excludes fees and interest.

Name:      Average Daily Balance
SQL:       SUM(end_of_day_balance) /
            COUNT(DISTINCT balance_date)
Synonyms:  ADB, average balance, mean balance
Instruction: Use for balance inquiries. Calculated over
            the period specified by the user.

Name:      Churn Rate
SQL:       COUNT(DISTINCT CASE WHEN status = 'CLOSED'
            THEN account_id END) * 100.0 /
            NULLIF(COUNT(DISTINCT account_id), 0)
Synonyms:  attrition, account closure rate
Instruction: Percentage of accounts closed in the period.
```



SQL Expressions — Your Most Powerful Tool

Filters — Boolean Conditions

```
Name:      Active Accounts
SQL:       account_status = 'ACTIVE'
Synonyms:  current accounts, open accounts
Instruction: Apply when users ask about active or
             current accounts. Excludes CLOSED,
             SUSPENDED, PENDING_REVIEW.

Name:      Recent Transactions
SQL:       transaction_date >= DATEADD(DAY, -30,
             CURRENT_DATE())
Synonyms:  last 30 days, recent, this month
Instruction: Default time filter when user says 'recent'
             without specifying an exact date range.

Name:      High Value
SQL:       total_relationship_value > 1000000
Synonyms:  premium, high net worth, HNW, large accounts
Instruction: Apply when users refer to high-value,
             premium, or large customers/accounts.
```



SQL Expressions — Your Most Powerful Tool

Dimensions — Row-Level Transformations

Name: Customer Segment

SQL:

```
CASE
    WHEN total_assets > 10000000 THEN 'UHNW'
    WHEN total_assets > 1000000 THEN 'HNW'
    WHEN total_assets > 250000 THEN 'Affluent'
    ELSE 'Mass Market'
END
```

Synonyms: segment, tier, customer type

Instruction: Use when users ask to break down by customer segment or tier.

Name: Fiscal Quarter

SQL:

```
CONCAT('FY', YEAR(DATEADD(MONTH, 3, date))),
'-Q', QUARTER(DATEADD(MONTH, 3, date)))
```

Synonyms: quarter, fiscal quarter, FQ

Instruction: Our fiscal year starts in October.
FY2026-Q1 = Oct-Dec 2025.



Example SQL Queries — Gold Standard Patterns

Example queries teach Genie how to handle complex, multi-step questions. Each query should have a title phrased as a user would ask the question and usage guidance explaining when to apply it.

Pattern 1: Complex Aggregation with Business Logic

```
-- Title: What are our top 10 most expensive jobs this month?  
-- Usage Guidance: Use when users ask about expensive  
-- jobs, costly workloads, or top cost contributors.
```

```
SELECT  
  j.job_name,  
  w.workspace_name,  
  j.job_owner AS owner,  
  COUNT(DISTINCT r.run_id) AS total_runs,  
  ROUND(SUM(b.usage_quantity * b.list_price), 2)  
    AS total_cost_usd,  
  ROUND(AVG(r.duration_seconds) / 60, 1)  
    AS avg_duration_min  
FROM gold.finops.job_runs r  
JOIN gold.finops.jobs j ON r.job_id = j.job_id  
JOIN gold.finops.billing_usage b ON r.run_id = b.run_id  
JOIN gold.finops.workspaces w ON r.workspace_id = w.id  
WHERE b.usage_date >= DATE_TRUNC('month', CURRENT_DATE())  
GROUP BY j.job_name, w.workspace_name, j.job_owner  
ORDER BY total_cost_usd DESC  
LIMIT 10;
```



Example SQL Queries — Gold Standard Patterns

Pattern 2: Parameterized Query (Becomes Trusted Asset)

```
-- Title: Show cost breakdown for a specific workspace
-- Parameter: workspace_name (STRING)
-- Usage Guidance: Use when user asks about costs for
--   a specific workspace by name.

SELECT
    sku_name,
    DATE_TRUNC('week', usage_date) AS week,
    SUM(usage_quantity) AS total_dbus,
    ROUND(SUM(usage_quantity * list_price), 2) AS cost_usd
FROM gold.finops.billing_usage
WHERE workspace_name = :workspace_name
    AND usage_date >= DATEADD(MONTH, -3, CURRENT_DATE())
GROUP BY sku_name, DATE_TRUNC('week', usage_date)
ORDER BY week DESC, cost_usd DESC;
```

RESULT: Parameterized queries become **Trusted Assets**. When Genie uses the exact template and fills in parameters, the response is labeled 'Trusted' — giving users extra confidence in the answer.



Example SQL Queries — Gold Standard Patterns

Pattern 3: Month-over-Month Comparison

```
-- Title: Show month-over-month cost change by SKU
-- Usage Guidance: Use for cost trend analysis, MoM
-- comparisons, or 'how has cost changed' questions.

WITH monthly AS (
    SELECT
        sku_name,
        DATE_TRUNC('month', usage_date) AS month,
        SUM(usage_quantity * list_price) AS cost
    FROM gold.finops.billing_usage
    WHERE usage_date >= DATEADD(MONTH, -6, CURRENT_DATE())
    GROUP BY 1, 2
)
SELECT
    sku_name,
    month,
    ROUND(cost, 2) AS cost_usd,
    ROUND(LAG(cost) OVER
        (PARTITION BY sku_name ORDER BY month), 2) AS prev,
    ROUND((cost - LAG(cost) OVER
        (PARTITION BY sku_name ORDER BY month)) /
        NULLIF(LAG(cost) OVER
        (PARTITION BY sku_name ORDER BY month), 0)
        * 100, 1) AS pct_change
FROM monthly
ORDER BY sku_name, month DESC;
```



Advanced Curation Techniques

—

Trusted Assets via SQL Functions

For calculations that must never vary, register them as Unity Catalog functions. Genie calls the function instead of generating SQL — the response is labeled Trusted.

```
CREATE OR REPLACE FUNCTION gold.finops.workspace_cost(  
  p_workspace STRING DEFAULT NULL,  
  p_months INT DEFAULT 3  
) RETURNS TABLE  
COMMENT 'Returns cost breakdown by SKU for a workspace.  
Parameters:  
- p_workspace: workspace name (e.g., prod-analytics).  
  If NULL, returns all workspaces.  
- p_months: lookback period in months (default 3).  
Example questions:  
- What did prod-analytics cost last quarter?  
- Show all workspace costs for the past 6 months'  
RETURN  
  SELECT  
    workspace_name,  
    sku_name,  
    DATE_TRUNC('month', usage_date) AS month,  
    ROUND(SUM(usage_quantity * list_price), 2) AS cost  
  FROM gold.finops.billing_usage  
 WHERE (isnull(p_workspace)  
        OR workspace_name = p_workspace)  
        AND usage_date >= DATEADD(  
          MONTH, -p_months, CURRENT_DATE())  
 GROUP BY 1, 2, 3  
 ORDER BY month DESC, cost DESC;
```

ACTION: Include DEFAULT NULL for optional parameters and use isnull() checks. Include example questions in the COMMENT — Genie reads these to determine when to invoke the function.



Clarification Instructions Pattern

Teach Genie to ask clarifying questions instead of guessing.

```
# In General Instructions:
```

```
When users ask about cost analysis but don't specify  
a time range, workspace, or SKU type, you must ask  
a clarification question first. Example:
```

```
'To analyze costs, please specify:
```

- 1) Time range (e.g., last month, Q1 2026)
- 2) Workspace (optional – defaults to all)
- 3) SKU type (optional – defaults to all)'

```
When users ask about 'performance' without specifying  
whether they mean job performance, query performance,  
or cluster performance, ask:
```

```
'Which type of performance are you interested in?
```

- Job performance (duration, success rate)
- Query performance (latency, throughput)
- Cluster performance (utilization, scaling)'



General Instructions Template

Keep general instructions focused on behavioral guidance and domain context.

PURPOSE

This space provides cost and usage analytics for our Databricks platform. Target audience: platform engineers and FinOps analysts.

KEY ENTITIES

- A workspace belongs to a business unit and team
- Jobs run on clusters and consume DBUs
- Cost = usage_quantity * list_price (in USD)
- DBU = Databricks Unit, the unit of compute consumption

BUSINESS RULES

- Fiscal year starts in October (FY2026 = Oct 2025-Sep 2026)
- 'Active workspace' = at least one job run in last 30 days
- Default time range for cost queries = current month
- Round dollar amounts to 2 decimal places
- Round percentages to 1 decimal place

CLARIFICATIONS (ask when ambiguous)

[Add clarification instructions here]

SUMMARY FORMAT

Instructions you must follow when providing summaries:

- Cite the specific table and column names used
- Use bullet points for multi-part answers
- Include the date range covered in results
- State the total count when showing top-N results



Benchmark-Driven Development

—

The Proven Accuracy Progression

Field testing across enterprise deployments has established a repeatable accuracy progression

Iteration	Changes	Accuracy	Effort	Key Insight
0 Baseline	No curation, poor names	0%	—	AI cannot guess your schema
1 Naming	Clear table + column names	~30%	Low	Single biggest ROI action
2 Metadata	+ descriptions, PK/FK	~45%	Medium	Multi-table queries start working
3 Values	+ entity matching	~55%	Low	Filters match actual data values
4 SQL Examples	+ 10-15 example queries	~77%	Medium	Largest single accuracy jump
5 Instructions	+ text instructions	~90-100%	Low	Fills remaining edge cases

RESULT: Iteration 4 (adding example SQL queries) typically delivers the **largest single accuracy improvement** — often +20 percentage points. One well-crafted example query can solve an entire category of similar questions simultaneously.



Building a Benchmark Suite

Step 8.1: Gather Questions from SMEs

Collect 20–50 representative questions. Cover all complexity levels:

Complexity	Example	Count Target
Simple lookup	What is the total cost for workspace X?	5-10
Aggregation	Show monthly cost trends for the last 6 months	5-10
Multi-table join	Which teams have the most expensive jobs?	5-8
Filtered	Show all failed jobs in the last week	5-8
Complex analytics	Month-over-month cost change by SKU	5-8
Ambiguous / edge case	Which workspaces are most expensive? (needs clarification)	3-5



Building a Benchmark Suite

Step 8.2: Write Ground Truth SQL

For each question, write the expected SQL. This is your reference answer.

Step 8.3: Add Multiple Phrasings

Question 1a: What are our top 10 most expensive jobs?

Question 1b: Show me the costliest workloads

Question 1c: Which jobs cost the most this month?

Question 1d: Top spending jobs

→ All four use the SAME reference SQL



Building a Benchmark Suite

Step 8.4: Run, Measure, Iterate

```
# Programmatic benchmark execution
from databricks.sdk import WorkspaceClient
w = WorkspaceClient()
# Get all benchmark question IDs

space = w.genie.get_space(
    space_id='<space_id>',
    include_serialized_space=True
)
q_ids = [q['id'] for q in
    space.serialized_space['benchmarks']['questions']]
# Run evaluation
eval_run = w.genie.genie_create_eval_run(
    space_id='<space_id>',
    benchmark_question_ids=q_ids
)
# Check results
results = w.genie.genie_list_eval_results(
    space_id='<space_id>',
    eval_run_id=eval_run.id
)
for r in results:
    print(f'{r.question}: {r.score}')
```

ACTION: Run benchmarks after every curation change. Always check for regressions — fixing one category of questions can sometimes break another. The benchmark suite is your safety net.



Sandbox → Production Playbook

—

Four-Stage Rollout

Stage	Users	Duration	Exit Criteria	Key Activities
1. Sandbox	1-2 curators	2-4 weeks	≥80% benchmark accuracy	Data prep, curation, benchmarking
2. UAT	3-5 SMEs	2-3 weeks	SME approval, <5% false answers	Feedback collection, curation refinement
3. Limited Prod	10-20 users	2-4 weeks	Stable accuracy, no major issues	Monitoring, response time optimization
4. Full Prod	All target users	Ongoing	Monthly benchmark reviews	Continuous improvement cadence



CI/CD with the Management API

Export Space Configuration

```
# Export space as JSON (for version control)
import json
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
space = w.genie.get_space(
    space_id='<dev_space_id>',
    include_serialized_space=True
)

# Save to file (commit to git)
with open('genie_space_config.json', 'w') as f:
    json.dump({
        'title': space.title,
        'description': space.description,
        'serialized_space': space.serialized_space
    }, f, indent=2)
```



CI/CD with the Management API

Deploy to Production

```
# Deploy from version-controlled config
import json
from databricks.sdk import WorkspaceClient

w = WorkspaceClient(profile='production')

with open('genie_space_config.json') as f:
    config = json.load(f)

# Create or update production space
prod_space = w.genie.create_space(
    title=config['title'],
    description=config['description'],
    warehouse_id='<prod_warehouse_id>',
    serialized_space=config['serialized_space']
)
print(f'Deployed: {prod_space.space_id}')
```



Cloning for Testing

Copied: Tables, settings, general instructions, example SQL queries, SQL functions.

NOT copied: Chat threads, monitoring data, benchmarks results.

ACTION: Use cloning for A/B testing curation changes. Clone the production space, apply changes to the clone, run benchmarks, compare accuracy before promoting.



Use GenieRails – ABAC + UI to Production

1. Import your existing Genie Space configuration into code (instructions, benchmarks, SQL measures — all captured verbatim)
2. Generate ABAC governance: groups, tag policies, column masking, row filters, catalog grants
3. Review and tune the generated governance
4. Apply everything to your dev workspace
5. Promote the whole setup — governance and Genie config — to production



Refer to

[databricks-solutions/genierails](#)
[GenieRails – From UI to Production](#)
[End-to-end demo](#)

AVOID: Entity matching is **automatically disabled** for tables with row filters or column masks. This prevents sensitive values from being cached in the knowledge store. Impact: Genie may fail to match user terms to filtered data values. Workaround: Add explicit text instructions listing key categorical values.

Production Monitoring Cadence

Frequency	Actions
Daily (automated)	Monitor API error rates, response times, warehouse utilization
Weekly	Review Monitoring tab: top failing questions, flagged conversations, feedback trends
Bi-weekly	Re-run full benchmark suite; investigate any accuracy drops >5%
Monthly	Add new benchmark questions from real user queries; refresh entity matching; review SQL expressions
Quarterly	SME review of space purpose and coverage; assess if domain decomposition needs adjustment



Operational Runbook

Monitoring Tab — What to Watch

Access via the Monitoring tab (CAN MANAGE required). Focus on:

Signal	Action	Priority
Question with thumbs-down	Investigate: wrong SQL, wrong data, or user misunderstanding?	High
'Request Review' flag	Respond within 48 hours; add curation if needed	High
Repeated similar questions	Add as example SQL query if not already covered	Medium
Questions with no SQL generated	Genie lacked context — add metadata, synonyms, or examples	Medium
Questions outside intended domain	Add clarification instruction or update space description	Low



Feedback-to-Curation Loop

1. User asks question → Genie responds
2. User marks response as incorrect (thumbs down)
3. Space manager sees it in Monitoring tab
4. Manager investigates:
 - Was the SQL wrong? → Fix via example query
 - Was the data wrong? → Fix at data layer
 - Was the question ambiguous? → Add clarification
5. Manager adds the CORRECT query as an example
6. Manager re-runs benchmarks to verify fix
7. Improvement is permanent for all future users

AVOID: Feedback does NOT automatically change Genie behavior. Every improvement requires manual curation by the space manager. Budget 2-4 hours per week for active spaces.



Quick-Reference Cheat Sheets

—

Limits at a Glance

Resource	Limit
Tables per space	30
Total instructions (SQL queries + functions + text)	100
SQL expressions (measures + filters + dimensions)	200
Benchmark questions	500 per space
Entity matching columns	120 per space
Values per entity matching column	1,024
Value length (entity matching)	127 characters
UI throughput	20 questions/min/workspace
API throughput (free tier)	5 questions/min/workspace
Conversations per space	10,000
Messages per conversation	10,000
Query result rows	5,000
CSV download	~1 GB
String field length	25,000 characters
Text instruction blocks	1 per space



Curation Decision Flowchart

Benchmark question fails

- |
- ■ Wrong table selected?
 - | ■ ■ Fix: Table descriptions in UC
 - |
- ■ Wrong column selected?
 - | ■ ■ Fix: Column descriptions + synonyms
 - |
- ■ Wrong join path?
 - | ■ ■ Fix: PK/FK constraints + Joins tab
 - |
- ■ Wrong filter values?
 - | ■ ■ Fix: Entity matching + value dictionaries
 - |
- ■ Wrong calculation logic?
 - | ■ ■ Fix: Example SQL query with usage guidance
 - |
- ■ Calculation must be exact every time?
 - | ■ ■ Fix: SQL function (Trusted Asset)
 - |
- ■ Missing business context?
 - | ■ ■ Fix: SQL expression (measure/filter/dim)
 - |
- ■ None of the above?
 - ■ Fix: General text instruction (last resort)



References

—

Databricks Official Documentation

- [1] What is a Genie Space — <https://docs.databricks.com/aws/en/genie/>
- [2] Curate an Effective Genie Space (Best Practices) — <https://docs.databricks.com/aws/en/genie/best-practices>
- [3] Set Up and Manage a Genie Space — <https://docs.databricks.com/aws/en/genie/set-up>
- [4] Build a Knowledge Store — <https://docs.databricks.com/aws/en/genie/knowledge-store>
- [5] Use Trusted Assets in Genie Spaces — <https://docs.databricks.com/aws/en/genie/trusted-assets>
- [6] Genie Conversation API — <https://docs.databricks.com/aws/en/genie/conversation-api>
- [7] Genie Spaces with Dashboards — <https://docs.databricks.com/aws/en/dashboards/genie-spaces>
- [8] Genie REST API Reference — <https://docs.databricks.com/api/workspace/genie>
- [9] Databricks Python SDK (Genie) — <https://databricks-sdk-py.readthedocs.io/en/latest/workspace/dashboards/genie.html>



Databricks Blog Posts

[10] 'From Data to Dialogue: A Best Practices Guide for Building High-Performing Genie Spaces' — Databricks Blog, Feb 5, 2026

[11] 'How to Build Production-Ready Genie Spaces, and Build Trust Along the Way' — Databricks Blog, Feb 6, 2026

[12] 'AI/BI Genie Now Generally Available' — Databricks Blog, 2025

[13] 'Genie Conversation APIs: Public Preview' — Databricks Blog, 2025



Community & Field Engineering

[14] 'Best Practices for AI/BI Genie Spaces' — Medium / DBSQL SME Engineering

[15] 'Leverage Multiple Genie Spaces Inside an App Using the API' — Databricks Community Technical Blog

[16] 'AI/BI Genie Compound AI System' — Medium / Technology Hits

[17] Databricks Field Engineering — internal deployment patterns and benchmarking data across enterprise customers

[18] DAIS 2026 — Enterprise FinOps Genie deployment across 1,000+ workspaces (financial services reference implementation)

